

Training neural networks

- Gradient Descent.
- How MNIST hidden layers work
- Mnist recap:
 - digits on 28×28 pixel
 - each pixel bw 0 & 1
 - 784 pixels determine activation of 784 neurons in input layer of the network
 - activation of each neuron in following layers $\Rightarrow$ weighted sum of all activations in previous layer + bias

→ then apply σ or ReLU

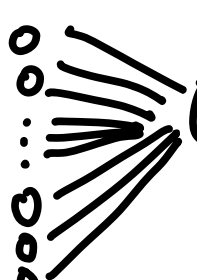
→ network has 13k weights
& biases.

learning = finding right weights
& biases

brightest ^{of 10} neurons in final layer = predicted digit

training: given images of handwritten
digits & correct labels, it'll
adjust 13k parameters to get
better at training data

- goal is model to generalize beyond training data.
- then u test it on more data to see performance
- it's all calculus. comes down to finding minima of a $C(x)$.
- each neuron connected to all neurons in the previous layer



A diagram showing a single neuron (represented by a circle) receiving multiple inputs from a previous layer (represented by a vertical column of circles). Lines connect each input circle to the neuron circle.

$$O = \sigma(w_1 a_1 + \dots + w_n a_n + b)$$

- weights = strength of connection
- bias = is connection usually active
- init all weights & biases randomly
- random network outputs a mess
- define a cost function

0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
garbage	correct



what is the
cost of this
difference?

- cost $f(x)$ says no this is a bad answer.

- cost here: add square of diffs b/w bad input & right input. that's cost of one training example

- cost small when prediction right

- avg cost of 10k+ training examples?

- that's measure of network perf

- network itself is basically a function
- input: 784 ~~xs~~ (pixels)
- output: 10 ~~xs~~

Parameters: 13,002 weights & biases

cost function

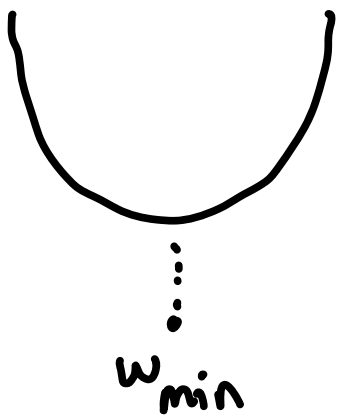
input: 13,002 weights & biases

output: 1 ~~x~~ number, cost, how bad weights & biases are

Parameters: many training examples

- need to tell it how to change weights & biases so it gets better

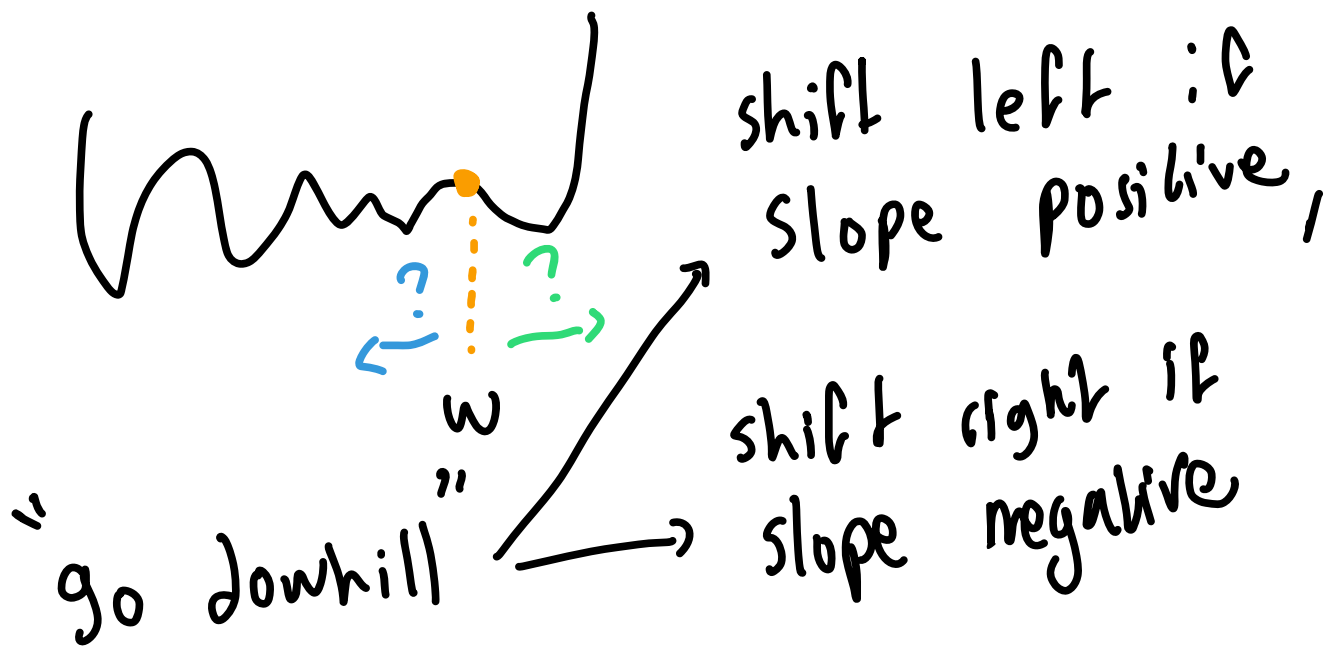
- $C(w_1, \dots, w_{13002}) \rightarrow C(w)$
single input



How to find input, that minimizes value of the cost function

$$\frac{dC}{dw}(w) = 0 \quad \text{sure but hard}$$

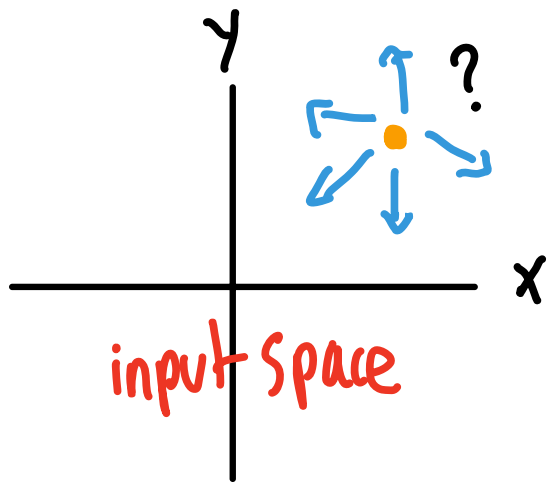
- instead start w/ any old input then figure out which way you should step, to make output lower.



- you'll find local minima
- for lots of inputs, you'll find all the valleys, the local minima

- local mins easy to find
- global min is hard
- make step size proportional to slope. so smaller steps if the slope is getting flatter. Helps u from overshooting.
- imagine $f(x)$ w/ 2 inputs & 1 output, in 3D space.
- input space is (x, y) plane

- cost function is surface above it
- instead of asking slope, you ask which direction to step decreases $C(x,y)$ most quickly



what's the downhill direction?

From multivariable calculus, gradient of function gives direction of steepest ascent

$$\nabla C(x,y)$$

which direction to increase $C(x,y)$ most quickly

negative of gradient gives a
direction that decreases function
most quickly

length of gradient vector tells
you how steep steepest slope is

there exists a way to compute
this vector

Algorithm for minimizing $C(x, y)$

= Compute ∇C
small step downhill in $-\nabla C$ direction
repeat.

Same idea but func w/ 13,002 inputs.

Organize all 13k w & b's in col vector

$$\vec{w} = \begin{bmatrix} w \\ b \end{bmatrix}, \quad -\nabla C(\vec{w}) = \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}$$

13k weights
& biases

How to nudge
all weights &
biases, which
nudges to largest
decrease of the
cost function.

• Changing w & b to decrease $C(\dots)$,
= making neural output less like random, more
decision.

- Algorithm for computing gradient efficiently = backprop.
what happens to each weight & bias for each piece of training data?

- network learning = min cost

$f()$ (gives cost of network prediction vs. correct)

- cost $f(x)$ needs smooth output to find little steps downhill

- this is why perceptrons are range b/w $[0-1]$, not binary
- nudging $E()$ by multiple of negative gradient = gradient descent
- converge onto local minima of cost function (valley in graph)
- each component of $-\nabla C(\vec{w})$

tells us :

$$\rightarrow \vec{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix} \quad -\nabla C(\vec{w}) = \begin{bmatrix} +x \\ -y \end{bmatrix}$$

w_0 should inc by x
 w_1 should dec by y

- adjust to 1 weight might have greater input to $\text{cost}()$
- Some corrections matter more for training data
- gradient vector encodes importance of each weight & bias.
- which change is most bang for buck

ex:

$$C(x, y) = \frac{3}{2}x^2 + \frac{1}{2}y^2$$

$$\nabla C(1, 1) = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

changes to x 3 times more important

- recap:

- network itself is basically a function
- input: 784 ~~xs~~ (pixels)
- output: 10 ~~xs~~

Parameters: 13,002 weights & biases

cost function

input: 13,002 weights & biases

output: 1 ~~x~~ number, cost, how bad weights & biases are

Parameters: many training examples

Gradient of cost function
is another layer.

what nudges to all weights

if bins cause fastest
change to val of

Cost Function. which

Changes to which

weights matter the most?

When init network random,
then train w/ gradient
descent,

How well perform on ^{new} images

- State of the art is 99.84%
- 99.5% is baseline accuracy.

- What happens if
feed in random image?
→ confident nonsense

- multilayer perceptrons
is old tech research
in 80s & 90s.

. Read Michael Nielsen's
book on Neural
networks